EXP 1 - Simple Linear Regression

```
# 1. Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import
    train_test_split
from sklearn.linear_model import
    LinearRegression
from sklearn.metrics import
    mean_squared_error, r2_score
# 2. Read CSV file
data = pd.read_csv('data.csv')
# 3. Display dataset
print(data.head())
# 4. Separate input and output
X = data[['Hours']]
y = data['Scores']
# 5. Split dataset
X_train, X_test, y_train, y_test =
    train_test_split(
    X, y, test_size=0.2,
        random_state=42
)
# 6. Create model
model = LinearRegression()
# 7. Train model
model.fit(X_train, y_train)
# 8. Predict
y_pred = model.predict(X_test)
# 9. Evaluate model
print("Slope:", model.coef_)
print("Intercept:", model.intercept_)
print("Mean Squared Error:",
    mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test,
    y_pred))
# 10. Visualize
plt.scatter(X, y, color='blue',
    label='Actual Data')
plt.plot(X, model.predict(X),
    color='red', label='Regression
    Line')
plt.xlabel('Hours')
plt.ylabel('Scores')
plt.title('Linear Regression')
plt.legend()
plt.show()
```

EXP 2(a) - Histogram Equalization

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("jommuFLOWER.jpg")
img_rgb = cv2.cvtColor(img,
    cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img,
    cv2.COLOR_BGR2GRAY)
equalized = cv2.equalizeHist(gray)
plt.figure(figsize=(15, 10))
plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Colour Image")
plt.axis("off")
plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.subplot(2, 3, 3)
plt.hist(gray.ravel(), 256, [0, 256],
    color="black")
plt.title("Histogram of Grayscale
    Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.subplot(2, 3, 5)
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalized Image")
plt.axis("off")
plt.subplot(2, 3, 4)
plt.axis("off")
plt.subplot(2, 3, 6)
plt.hist(equalized.ravel(), 256, [0,
    256], color="black")
plt.title("Histogram of Equalized
    Image")
plt.show()
```

EXP 2(b) - Morphological Opening and Closing

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread("jommuFLOWER.jpg")
img_rgb = cv2.cvtColor(img,
    cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img,
    cv2.COLOR_BGR2GRAY)
kernel =
    cv2.getStructuringElement(cv2.MORPH
    _RECT, (5, 5))
opening = cv2.morphologyEx(gray,
    cv2.MORPH_OPEN, kernel)
closing = cv2.morphologyEx(gray,
    cv2.MORPH_CLOSE, kernel)
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.imshow(img_rgb)
plt.title("Original Colour Image")
plt.axis("off")
plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.subplot(2, 2, 3)
plt.imshow(opening, cmap="gray")
plt.title("Morphological Opening
    Image")
plt.axis("off")
plt.subplot(2, 2, 4)
plt.imshow(closing, cmap="gray")
plt.title("Morphological Closing
    Image")
plt.axis("off")
plt.show()
```

CIFAR-10 classification.py

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import
    cifar10
from tensorflow.keras.models import
    Sequential
from tensorflow.keras.layers import
    Flatten, Dense
from tensorflow.keras.utils import
    to_categorical
(x_train, y_train), (x_test, y_test) =
    cifar10.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
class_names = [
    "Airplane",
    "Automobile",
    "Bird",
    "Cat",
    "Deer",
    "Dog",
    "Frog",
    "Horse",
    "Ship",
    "Truck"
]
def create_model(hidden_units,
    activation):
    model = Sequential([
        Flatten(input_shape=(32, 32,
            3)),
        Dense(hidden_units[0],
            activation=activation),
        Dense(hidden_units[1],
            activation=activation),
        Dense(hidden_units[2],
            activation=activation),
        Dense(10, activation="softmax")
    ])
    model.compile(
        optimizer="adam",
        loss="categorical_crossentropy"
            ,
        metrics=["accuracy"]
    )
    return model
hidden_units_list = [
    (1024, 512, 256),
    (512, 256, 128),
    (256, 128, 64),
    (128, 64, 32)
]
activation_list = [
    "relu",
    "tanh",
    "sigmoid"
]
results = []
best_acc = 0
best_model = None
best_config = None
for units in hidden_units_list:
    for act in activation_list:
        model = create_model(units,
            act)
        model.fit(x_train, y_train,
            epochs=5, batch_size=64,
            verbose=0)
        loss, acc =
            model.evaluate(x_test,
            y_test, verbose=0)
        results.append([*units, act,
            loss, acc * 100])
        if acc > best_acc:
            best_acc, best_model,
                best_config = acc,
                model, (units, act)
print("=" * 70)
print("MODEL COMPARISON")
print("=" * 70)
print(
    f"{'Hidden Layer 1':>15}"
    f"{'Hidden Layer 2':>15}"
    f"{'Hidden Layer 3':>15}"
    f"{'Activation':>12}"
    f"{'Test Loss':>12}"
    f"{'Test Accuracy (%)':>20}"
)
for result in results:
    print(
        f"{result[0]:>15}"
        f"{result[1]:>15}"
        f"{result[2]:>15}"
        f"{result[3]:>12}"
        f"{result[4]:>12.4f}"
        f"{result[5]:>20.2f}"
    )
print()
print("=" * 70)
print("BEST MODEL")
print("=" * 70)
print("Hidden Units :", best_config[0])
print("Activation   :", best_config[1])
print("Accuracy     :", f"{best_acc *
    100:.2f} %")
plt.figure(figsize=(15, 6))
for i in range(10):
    prediction =
        best_model.predict(x_test[i].re
        shape(1, 32, 32, 3), verbose=0)
    predicted_class =
        np.argmax(prediction)
    actual_class = np.argmax(y_test[i])
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i])
    plt.title(
        f"Pred: {class_names[predicted_
            class]}\n"
        f"Actual:
            {class_names[actual_class]}
            "
    )
    plt.axis("off")
plt.tight_layout()
plt.show()
```

ALGORITHM - EXP 1 - Simple Linear Regression

Aim: To implement simple linear regression to predict student scores from study hours and evaluate the model.

1. Start.
2. Import Pandas, NumPy, Matplotlib and the required Scikit-learn functions.
3. Read the dataset from `data.csv` using `read_csv()`.
4. Display the first five records using `head()`.
5. Separate the input feature `Hours` into X and target `Scores` into y.
6. Split the dataset into training and testing sets:
   a. Set test size to 0.2.
   b. Set random_state to 42 for reproducibility.
7. Create a LinearRegression model.
8. Train the model using X_train and y_train.
9. Predict scores for X_test.
10. Evaluate the model:
    a. Display the slope and intercept.
    b. Calculate Mean Squared Error.
    c. Calculate the R2 score.
11. Plot actual data points and the regression line.
12. Add axis labels, title and legend, then display the plot.
13. Stop.

ALGORITHM - EXP 2(a) - Histogram Equalization

Aim: To enhance the contrast of a grayscale image using histogram equalization and compare the original and equalized histograms.

1. Start.
2. Import OpenCV and Matplotlib.
3. Read the input colour image `jommuFLOWER.jpg`.
4. Convert the image from BGR to RGB for correct colour display.
5. Convert the original BGR image to grayscale.
6. Apply histogram equalization to the grayscale image using `equalizeHist()`.
7. Create a figure with a 2 x 3 subplot layout.
8. Display the original RGB image and grayscale image.
9. Calculate and display the grayscale image histogram using 256 intensity bins.
10. Display the histogram-equalized grayscale image.
11. Turn off the unused fourth subplot.
12. Calculate and display the equalized image histogram.
13. Display the complete figure.
14. Stop.

ALGORITHM - EXP 2(b) - Morphological Opening and Closing

Aim: To perform morphological opening and closing on a grayscale image using a rectangular structuring element.

1. Start.
2. Import OpenCV and Matplotlib.
3. Read the input image `jommuFLOWER.jpg`.
4. Convert the image from BGR to RGB for display.
5. Convert the original image to grayscale.
6. Create a rectangular structuring element of size 5 x 5.
7. Perform morphological opening on the grayscale image using the kernel.
8. Perform morphological closing on the grayscale image using the same kernel.
9. Create a figure with a 2 x 2 subplot layout.
10. Display the original colour image and grayscale image.
11. Display the opening result.
12. Display the closing result.
13. Turn off the image axes and display the complete figure.
14. Stop.

ALGORITHM - Exp 3: Comparing MLP architectures and activation functions on CIFAR-10

Aim: To build a multilayer perceptron with different hidden-layer sizes and activation functions on CIFAR-10, and find the best model.

Algorithm:

1. Start.
2. Import TensorFlow, Keras (Sequential, Flatten, Dense, to_categorical, cifar10) and Matplotlib.
3. Load the CIFAR-10 dataset (50,000 training and 10,000 test images of size 32x32x3, 10 classes).
4. Normalize the pixel values to the range 0 to 1 by dividing by 255.
5. Convert the class labels to one-hot vectors using to_categorical.
6. Define the 10 class names (Airplane, Automobile, ..., Truck).
7. Initialize best = None and best_acc = 0.
8. For each hidden-layer configuration in {(128,64,32), (256,128,64), (1024,512,256)}:
   a. For each activation function in {relu, tanh, sigmoid}:
      i. Build a Sequential model: Flatten layer, then three Dense hidden layers with the chosen units and activation, then Dense(10, softmax).
      ii. Compile the model with the Adam optimizer, categorical cross-entropy loss and accuracy metric.
      iii. Train for 5 epochs with batch size 64.
      iv. Evaluate on the test data to get the loss and accuracy.
      v. Print the configuration, activation, loss and accuracy.
      vi. If accuracy > best_acc, set best to this model and update best_acc.
9. Select test image number 100 and predict its class using the best model (argmax of the output probabilities).
10. Print the predicted class and actual class, and display the image.
11. Stop.

cifar10_with&without_initialization.py

```
import pandas as pd
from tensorflow.keras.datasets import
    cifar10
from tensorflow.keras.models import
    Sequential
from tensorflow.keras.layers import
    Flatten, Dense, Dropout
from tensorflow.keras.regularizers
    import l2
from tensorflow.keras.utils import
    to_categorical
# Load and prepare data
(x_train, y_train), (x_test, y_test) =
    cifar10.load_data()
x_train, x_test = x_train / 255.0,
    x_test / 255.0
y_train, y_test =
    to_categorical(y_train),
    to_categorical(y_test)
def build_model(init, dropout, reg):
    model = Sequential([Flatten(input_sha
        pe=(32, 32, 3))])
    for units in [512, 256, 64]:
        model.add(Dense(units,
            activation="relu",
            kernel_initializer=init,
            kernel_regularizer=reg))
        if dropout:
            model.add(Dropout(0.5))
    model.add(Dense(10,
        activation="softmax",
        kernel_initializer=init))
    return model
# (name, initializer, dropout,
    regularizer, regularization label)
experiments = [
    ("Baseline",
        "glorot_uniform", False, None,
        "None"),
    ("Xavier",
        "glorot_uniform", False, None,
        "None"),
    ("Kaiming",          "he_normal",
        False, None,       "None"),
    ("Dropout",
        "glorot_uniform", True,  None,
        "Dropout"),
    ("L2 Regularization",
        "glorot_uniform", False,
        l2(0.001),  "L2"),
]
results = []
for name, init, dropout, reg, reg_label
    in experiments:
    for opt in ["Adam", "SGD"]:
        model = build_model(init,
            dropout, reg)
        model.compile(optimizer=opt.lower
            (), loss="categorical_crossen
            tropy", metrics=["accuracy"])
        model.fit(x_train, y_train,
            epochs=5, batch_size=64,
        validation_data=(x_test, y_test))
        loss, acc =
            model.evaluate(x_test,
            y_test, verbose=0)
        results.append([name, opt,
            reg_label, loss, acc * 100])
results_df = pd.DataFrame(results,
    columns=[
    "Model", "Optimizer",
        "Regularization",
    "Test Loss", "Test Accuracy (%)"
])
print("\n" + "=" * 100)
print("FINAL COMPARISON")
print("=" * 100)
print(results_df.to_string(index=False))
```

MNIST classification using CNN.py

```
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import
    mnist
from tensorflow.keras.models import
    Sequential
from tensorflow.keras.layers import
    Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.utils import
    to_categorical
# Load and prepare data
(x_train, y_train), (x_test, y_test) =
    mnist.load_data()
x_train = x_train.reshape(-1, 28, 28, 1)
    / 255.0
x_test = x_test.reshape(-1, 28, 28, 1) /
    255.0
y_train, y_test =
    to_categorical(y_train),
    to_categorical(y_test)
def build_model(extra_conv):
    model = Sequential([
        Conv2D(32, (3, 3),
            activation="relu",
            input_shape=(28, 28, 1)),
        MaxPooling2D((2, 2))
    ])
    if extra_conv:
        model.add(Conv2D(64, (3, 3),
            activation="relu"))
        model.add(MaxPooling2D((2, 2)))
    model.add(Flatten())
    model.add(Dense(128,
        activation="relu"))
    model.add(Dense(10,
        activation="softmax"))
    return model
# Model 1 has one conv layer, Model 2
    has two
for number, extra_conv in [(1, False),
    (2, True)]:
    model = build_model(extra_conv)
    model.compile(optimizer="adam",
        loss="categorical_crossentropy",
        metrics=["accuracy"])
    model.fit(x_train, y_train,
        epochs=5, batch_size=64)
    loss, accuracy =
        model.evaluate(x_test, y_test,
        verbose=0)
    pred = model.predict(x_test[:2])
    print("\n============================
        ==")
    print(f"MODEL {number} RESULTS")
    print("============================
        ")
    print("Accuracy:", accuracy * 100,
        "%")
    print("Loss:", loss)
    plt.figure(figsize=(12,6))
    for i in range(2):
        one_hot=str(ytest[i].astype(int).
            tolist())
        plt.subplot(1,2,i+1)
        plt.imshow(xtest[i],cmap='gray')
        plt.title(f"Predictecd:{pred[i].a
            rgmax()}\n"
        f"Actual :{ytest[i].argmax()}\n"
        f"One hot:{one_hot}")
        plt.axis("off")
     plt.show()
```

VGG19_Mnist.py

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from keras.applications import VGG19
from keras.datasets import mnist
from keras.utils import to_categorical
from keras.models import Sequential,
    Model
from keras.layers import Dense, Flatten,
    Input
from keras.optimizers import Adam
(X_train, y_train), (X_test, y_test) =
    mnist.load_data()
# Load a subset of the dataset (5000
    training images and 500 testing
    images)
X_train, y_train = X_train[:5000],
    y_train[:5000]
X_test, y_test = X_test[:500],
    y_test[:500]
X_train_froz, X_test_froz = X_train /
    255.0, X_test / 255.0
y_train_froz, y_test_froz =
    to_categorical(y_train),
    to_categorical(y_test)
# Fixed feature extractor approach
model_frozen = Sequential([
Flatten(input_shape=X_train.shape[1:]),
Dense(256, activation='relu'),
Dense(10, activation='softmax')
])
model_frozen.compile(optimizer=Adam(learn
    ing_rate=0.001),
    loss='categorical_crossentropy',
        metrics=['accuracy'])
# Training the frozen model
history_frozen =
    model_frozen.fit(X_train_froz,
    y_train_froz, epochs=5,
        batch_size=32,
        validation_data=(X_test_froz,
        y_test_froz))
# Convert grayscale images to RGB (VGG
    requires 3 channels)
X_train_rgb =
    tf.image.grayscale_to_rgb(tf.expand_d
    ims(X_train, axis=-1))
X_test_rgb =
    tf.image.grayscale_to_rgb(tf.expand_d
    ims(X_test, axis=-1))
# Resize images to match VGG input size
    (224x224)
X_train_resized =
    tf.image.resize(X_train_rgb, (224,
    224))
X_test_resized =
    tf.image.resize(X_test_rgb, (224,
    224))
# Normalize pixel values to [0, 1]
X_train_resized = X_train_resized / 255.0
X_test_resized = X_test_resized / 255.0
# Convert labels to one-hot encoding
y_train_rgb = to_categorical(y_train,
    num_classes=10)
y_test_rgb = to_categorical(y_test,
    num_classes=10)
base_model = VGG19(weights='imagenet',
    include_top=False, input_shape=(224,
    224, 3))
# Add custom classification layers on top
x = Flatten()(base_model.output)
x = Dense(256, activation='relu')(x)
output = Dense(10,
    activation='softmax')(x)
model_finetuned =
    Model(inputs=base_model.input,
    outputs=output) # Create the
    fine-tuned model
# Freeze the layers of the base VGG model
for layer in base_model.layers:
    layer.trainable = False
model_finetuned.compile(optimizer=Adam(le
    arning_rate=0.001),
    loss='categorical_crossentropy',
        metrics=['accuracy'])
# Train the model
history_finetuned =
    model_finetuned.fit(X_train_resized,
    y_train_rgb, batch_size=64,
        epochs=5,
            validation_data=(X_test_resiz
            ed, y_test_rgb))
# Evaluate the model
loss, accuracy =
    model_finetuned.evaluate(X_test_resiz
    ed, y_test_rgb)
print("Test accuracy:", accuracy)
plt.plot(history_frozen.history['accuracy
    '], label='Fixed Feature Extractor
    (Train)')
plt.plot(history_frozen.history['val_accu
    racy'], label='Fixed Feature
    Extractor (Validation)')
plt.plot(history_finetuned.history['accur
    acy'], label='Fine-Tuning (Train)')
plt.plot(history_finetuned.history['val_a
    ccuracy'], label='Fine-Tuning
    (Validation)')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

ALGORITHM - Exp 4: Effect of initializers, regularizers, dropout and optimizers

Aim: To compare the performance and training time of a deep network on CIFAR-10 using different weight initializers, regularizers, dropout and optimizers.

Algorithm:

1. Start.
2. Import TensorFlow, Keras layers, regularizers (l1, l2), Pandas and time.
3. Load CIFAR-10, normalize it by dividing by 255 and one-hot encode the labels.
4. Define 7 configurations as (name, initializer, regularizer, dropout flag): Baseline, Xavier, Dropout, L1, L2, He Normal and Kaiming.
5. Create an empty list results.
6. For each configuration:
    a. For each optimizer in {Adam (lr = 0.001), SGD (lr = 0.01)}:
        i. Create a model with an Input layer of shape (32,32,3) and a Flatten layer.
        ii. For each layer size in [512, 256, 64]: Add a Dense layer with ReLU activation, the chosen initializer and the chosen regularizer. If the dropout flag is set, add a Dropout(0.5) layer.
        iii. Add the output layer Dense(10, softmax).
        iv. Compile with the chosen optimizer and categorical cross-entropy loss.
        v. Record the start time.
        vi. Train for 5 epochs with batch size 64, using the test set as validation data.
        vii. Evaluate the model to get the loss and accuracy.
        viii. Calculate the training time (current time minus start time).
        ix. Append (model name, optimizer, loss, accuracy %, time) to results.
7. Convert results into a Pandas DataFrame.
8. Print the full results table and the training-time comparison table.
9. Stop.

ALGORITHM - Exp 5: CNN on MNIST with different numbers of convolution layers

Aim: To build and compare convolutional neural networks with one and two convolution layers for handwritten digit recognition.

Algorithm:

1. Start.
2. Import TensorFlow, Keras layers (Conv2D, MaxPooling2D, Flatten, Dense), the MNIST dataset and Matplotlib.
3. Load the MNIST dataset (60,000 training and 10,000 test images of size 28x28, 10 classes).
4. Reshape the images to (28, 28, 1) and normalize them by dividing by 255.
5. Convert the labels to one-hot vectors.
6. Define two model designs: Model 1 with one conv layer (32 filters) and Model 2 with two conv layers (32 and 64 filters), all with 3x3 kernels.
7. For each model design:
    a. Create an empty Sequential model.
    b. For each conv layer in the design, add Conv2D (ReLU activation) followed by MaxPooling2D (2x2).
    c. Add a Flatten layer, then Dense(128, ReLU), then Dense(10, softmax).
    d. Compile with the Adam optimizer and categorical cross-entropy loss.
    e. Train for 5 epochs with batch size 64.
    f. Evaluate on the test data and print the accuracy and loss.
    g. Predict the first two test images.
    h. Plot each image with its predicted digit, actual digit and one-hot vector.
8. Stop.

ALGORITHM - Exp 6: Transfer learning with VGG19 on MNIST

Aim: To classify MNIST digits using a pretrained VGG19 network with a two-stage process of feature extraction followed by fine-tuning.

Algorithm:

1. Start.
2. Import TensorFlow, VGG19, preprocess_input, Keras layers and Matplotlib.
3. Load the MNIST dataset and take a subset of 5,000 training and 1,000 test samples.
4. Preprocess the images:
    a. Add a channel dimension and convert to float.
    b. Convert grayscale to RGB (3 channels).
    c. Resize to 64x64.
    d. Apply VGG19 preprocess_input.
5. Load VGG19 with ImageNet weights, include_top=False and input shape (64,64,3).
6. Build the model: VGG19 base, then GlobalAveragePooling2D, then Dense(128, ReLU), then Dropout(0.3), then Dense(10, softmax).
7. For stage = 1 and 2:
    a. Stage 1 (feature extraction): freeze the VGG19 base completely and set the learning rate to 0.001.
    b. Stage 2 (fine-tuning): unfreeze the base, keep all layers except the last 8 frozen, and set the learning rate to 0.00001.
    c. Compile with the Adam optimizer and sparse categorical cross-entropy loss.
    d. Train for 3 epochs with batch size 32 and a 20% validation split.
    e. Plot the training vs. validation accuracy curve and loss curve for the stage.
8. Stop.

NIFTY50_Prediction.py

```
!pip install yfinance tensorflow
    scikit-learn matplotlib
import yfinance as yf
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import
    MinMaxScaler
from sklearn.metrics import
    mean_absolute_error
from tensorflow.keras.models import
    Sequential
from tensorflow.keras.layers import
    Dense
# Download data directly
data = yf.download("^NSEI",
    period="5y", progress=False)
if hasattr(data.columns, "nlevels")
    and data.columns.nlevels > 1:
    data.columns =
        data.columns.get_level_values
        (0)
data = data[['Open', 'High', 'Low',
    'Close']].dropna()
data['Target'] =
    data['Close'].shift(-1)
data = data.dropna()
X = data[['Open', 'High', 'Low',
    'Close']].values
y = data[['Target']].values
# Split chronologically
split = int(len(X) * 0.8)
sx, sy = MinMaxScaler(),
    MinMaxScaler()
X_train = sx.fit_transform(X[:split])
X_test = sx.transform(X[split:])
y_train = sy.fit_transform(y[:split])
y_test = sy.transform(y[split:])
# Build and train ANN
model = Sequential([
    Dense(64, activation='relu',
        input_shape=(4,)),
    Dense(32, activation='relu'),
    Dense(1)
])
model.compile(optimizer='adam',
    loss='mse')
model.fit(X_train, y_train,
    epochs=30, batch_size=32,
    verbose=0)
# Predict and evaluate
pred = sy.inverse_transform(model.pre
    dict(X_test, verbose=0))
actual = sy.inverse_transform(y_test)
print("MAE:",
    mean_absolute_error(actual,
    pred))
# Plot results
plt.plot(actual, label='Actual')
plt.plot(pred, label='Predicted')
plt.title('NIFTY-50 Price
    Prediction')
plt.legend()
plt.show()
# Next trading day's prediction
latest =
    sx.transform(X[-1].reshape(1,
    -1))
next_price =
    sy.inverse_transform(model.predic
    t(latest, verbose=0))
print("Predicted next closing
    price:",
    round(float(next_price[0, 0]),
    2))
```

IMDB review classifaction using RNN.py

```
from tensorflow.keras.datasets
    import imdb
from tensorflow.keras.models import
    Sequential
from tensorflow.keras.layers import
    Embedding,Dense,SimpleRNN
from tensorflow.keras.preprocessing.s
    equence import pad_sequences
vocab_size=10000
max_length=200
(xtrain,ytrain),(xtest,ytest)=imdb.lo
    ad_data(num_words=vocab_size)
xtrain=pad_sequences(xtrain,maxlen=ma
    x_length)
xtest=pad_sequences(xtest,maxlen=max_
    length)
model=Sequential([Embedding(vocab_siz
    e,32,input_length=vocab_size),
        SimpleRNN(32),
        Dense(1,activation='sigmoid')
            ])
model.compile(optimizer='adam',loss='
    binary_crossentropy',metrics=['ac
    curacy'])
model.fit(xtrain,ytrain,epochs=5,batc
    h_size=64)
loss,accuracy=model.evaluate(xtrain,y
    train,batch_size=32)
print("loss ",loss ,"\n Accuracy
    ",accuracy)
pred=model.predict(xtest[:1])
print("postive" if pred>=0.5 else
    "negative",pred)
```

EXP 9 - Comparison of RNN, LSTM and GRU

```
import tensorflow as tf
import pandas as pd
import time
from tensorflow.keras.datasets
    import imdb
from tensorflow.keras.preprocessing.s
    equence import pad_sequences
from tensorflow.keras.models import
    Sequential
from tensorflow.keras.layers import
    Embedding, SimpleRNN, LSTM, GRU,
    Dense
# Parameters
vocab_size = 10000
max_length = 200
# Load IMDB dataset
(x_train, y_train), (x_test, y_test)
    = imdb.load_data(
    num_words=vocab_size
)
print("Training samples:",
    len(x_train))
print("Testing samples:",
    len(x_test))
# Padding
x_train = pad_sequences(
    x_train,
    maxlen=max_length,
    padding="post",
    truncating="post"
)
x_test = pad_sequences(
    x_test,
    maxlen=max_length,
    padding="post",
    truncating="post"
)
print("Training data shape:",
    x_train.shape)
print("Testing data shape:",
    x_test.shape)
# Build model
def build_model(model_type):
    model = Sequential([
        tf.keras.Input(shape=(max_len
            gth,)),
        Embedding(input_dim=vocab_siz
            e, output_dim=64)
    ])
    if model_type == "Simple RNN":
        model.add(SimpleRNN(64))
    elif model_type == "LSTM":
        model.add(LSTM(64))
    elif model_type == "GRU":
        model.add(GRU(64))
    model.add(Dense(1,
        activation="sigmoid"))
    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
# Compare architectures
architectures = ["Simple RNN",
    "LSTM", "GRU"]
results = []
for architecture in architectures:
    print("\nTraining model:",
        architecture)
    model = build_model(architecture)
    start_time = time.time()
    model.fit(
        x_train,
        y_train,
        epochs=6,
        batch_size=64,
        validation_split=0.2,
        verbose=1
    )
    training_time = time.time() -
        start_time
    # Evaluate model
    test_loss, test_accuracy =
        model.evaluate(
        x_test, y_test, verbose=0
    )
    results.append({
        "Model": architecture,
        "Test Accuracy (%)":
            round(test_accuracy *
            100, 2),
        "Test Loss":
            round(test_loss, 4),
        "Training Time (s)":
            round(training_time, 2)
    })
# Display comparison
results_df = pd.DataFrame(results)
print("\n" + "=" * 75)
print("RNN, LSTM AND GRU PERFORMANCE
    COMPARISON")
print("=" * 75)
print(
    f"{'Model':<15}"
    f"{'Test Accuracy (%)':>20}"
    f"{'Test Loss':>15}"
    f"{'Training Time (s)':>23}"
)
print("-" * 75)
for _, row in results_df.iterrows():
    print(
        f"{row['Model']:<15}"
        f"{row['Test Accuracy
            (%)']:>20.2f}"
        f"{row['Test Loss']:>15.4f}"
        f"{row['Training Time
            (s)']:>23.2f}"
    )
print("=" * 75)
# Final analysis
best_accuracy_model = results_df.loc[
    results_df["Test Accuracy
        (%)"].idxmax(), "Model"
]
lowest_loss_model = results_df.loc[
    results_df["Test
        Loss"].idxmin(), "Model"
]
print("\n" + "=" * 35)
print("FINAL ANALYSIS")
print("=" * 35)
print("Best Test Accuracy:",
    best_accuracy_model)
print("Lowest Test Loss:",
    lowest_loss_model)
print("=" * 35)
```

ALGORITHM - Exp 7: LSTM for NIFTY 50 stock price prediction

Aim: To predict the closing price of the NIFTY 50 index using an LSTM network on time-series data.

Algorithm:

1. Start.
2. Import NumPy, yfinance, Matplotlib, Keras (LSTM, Dense, Dropout, BatchNormalization, EarlyStopping) and scikit-learn (MinMaxScaler, MAE, MSE).
3. Download the NIFTY 50 (^NSEI) data from 2015-01-01 to 2026-01-01.
4. Flatten the column index if it is multi-level.
5. Select the Open, High, Low and Close columns and remove missing values.
6. Scale all values to the range 0 to 1 using MinMaxScaler.
7. Create sequences:
    a. Each input X is a window of 20 consecutive days (shape 20x4).
    b. Each target y is the scaled Close price of the next day.
8. Split the data chronologically: the first 80% for training and the remaining 20% for testing (no shuffling).
9. Build the model: LSTM(64), then Dropout(0.2), then BatchNormalization, then Dense(32, ReLU), then Dense(1).
10. Compile with the Adam optimizer and mean squared error loss.
11. Train for up to 50 epochs with batch size 32, using EarlyStopping (patience 5, restore best weights).
12. Predict the Close prices on the test data.
13. Inverse-transform the predicted and actual values back to real price scale.
14. Calculate and print the MAE and MSE.
15. Plot the full Close price history, the actual test prices, the predicted test prices and a vertical line at the train/test split.
16. Stop.

ALGORITHM - Exp 8: Sentiment analysis on IMDB using SimpleRNN

Aim: To classify IMDB movie reviews as positive or negative using a Recurrent Neural Network.

Algorithm:

1. Start.
2. Import the IMDB dataset, Keras layers (Embedding, SimpleRNN, Dense) and pad_sequences.
3. Load the IMDB dataset, keeping only the 10,000 most frequent words (25,000 training and 25,000 test reviews).
4. Pad or truncate every review to a fixed length of 200 words.
5. Build the model: Embedding(10000, 32), then SimpleRNN(32), then Dense(1, sigmoid).
6. Compile with the Adam optimizer, binary cross-entropy loss and accuracy metric.
7. Train for 5 epochs with batch size 64.
8. Evaluate on the test data and print the test loss and accuracy.
9. Predict the sentiment probability of the first test review.
10. If the probability is >= 0.5, print "Positive". Otherwise print "Negative".
11. Stop.

ALGORITHM - EXP 9 - Comparison of RNN, LSTM and GRU

Aim: To compare Simple RNN, LSTM and GRU models for binary sentiment classification on the IMDB movie-review dataset.

1. Start.
2. Import TensorFlow, Pandas, time, the IMDB dataset, sequence-padding utilities, model classes and recurrent layers.
3. Set vocabulary size to 10,000 and maximum review length to 200.
4. Load the IMDB dataset, retaining the 10,000 most frequent words.
5. Display the number of training and testing samples.
6. Pad or truncate training and testing reviews to 200 tokens:
    a. Use post-padding.
    b. Use post-truncation.
7. Display the shapes of the padded datasets.
8. Define a model-building function:
    a. Create a Sequential model with an input of length 200.
    b. Add an Embedding layer with output dimension 64.
    c. Add SimpleRNN(64), LSTM(64) or GRU(64), according to the selected model type.
    d. Add Dense(1) with sigmoid activation.
    e. Compile using Adam, binary cross-entropy and accuracy.
9. Define the architectures Simple RNN, LSTM and GRU; initialize an empty results list.
10. For each architecture:
    a. Build the corresponding model.
    b. Record the start time.
    c. Train for 6 epochs with batch size 64 and validation split 0.2.
    d. Calculate the training time.
    e. Evaluate the model on the test data.
    f. Store model name, test accuracy, test loss and training time.
11. Convert the results list into a Pandas DataFrame.
12. Print a formatted comparison table containing accuracy, loss and training time.
13. Identify and print the model with the highest test accuracy.
14. Identify and print the model with the lowest test loss.
15. Stop.